POLICY-AWARE

Code Compliance Auditor

An AI-powered hybrid system that scans codebases for business policy violations using CodeBERT, LLaMA 3.1, Kimi K2, Gemini Embeddings & FAISS

Rithvik
System Architect

Rishil
Pipeline & Integration

Rohan
Model Fine-Tuning

The Problem

Manual Review Fails at Scale

Large codebases have hundreds of functions. Human review misses violations — especially subtle ones like bypassing service layers or missing audit logs.

Policy Violations = Real Risk

GDPR non-compliance, unlogged transactions, hardcoded discounts — these cause regulatory fines, security breaches, and audit failures.

No Automated Policy Enforcement

Linters check syntax. No tool checks whether your code follows your company's own business rules and architecture policies.

Our Solution: A hybrid AI system that automatically audits code against business policies — at any scale.

Tech Stack

CLASSIFIER

CodeBERT (microsoft/codebert-base)

LLM SUMMARIZER

LLaMA 3.1 8B Instant via Groq

LLM AUDITOR

Kimi K2 Instruct (moonshotai) via Groq

EMBEDDINGS

Gemini Embedding 2 Preview (3072-dim)

VECTOR DB

FAISS (faiss-cpu) — semantic policy search

CODE PARSING

Python ast — Abstract Syntax Tree chunking

FINE-TUNING

HuggingFace transformers + PEFT + LoRA

REPORTS

ReportLab PDF · JSON · Google Colab GPU

End-to-End Workflow

1

GitHub Repo

Clone & find .py files

2

AST Chunking

Functions & classes

3

LLaMA Summary

2-3 sentence summary

4

FAISS Retrieval

Top-3 relevant policies

5

CodeBERT

p(violation) per policy

6

Hybrid Decision

>65% or <35% fast-path

7

Kimi K2 Audit

Structured JSON verdict

8

PDF Report

Downloadable audit

Compliance Policies

Payments

3

- AuditLogger on transactions
- Only approved PaymentService providers
- No direct DB updates to payment/order status

Discounts

2

- No hardcoded discounts anywhere
- All discounts via OfferService

Customer PII

3

- Email/data only via Customer service
- No direct DB queries for PII
- GDPR-compliant access patterns

Communications

2

- All emails through central service
- Opt-out preferences must be respected

Architecture

2

- No direct ORM model manipulation
- Critical ops must have structured logging

RAG Pipeline: Policy Retrieval

HOW FAISS RETRIEVAL WORKS

1. 12 policies → Gemini Embeddings (3072-dim RETRIEVAL_DOCUMENT mode) → stored in FAISS index
2. Each code chunk → LLaMA 8B summary → Gemini Embedding (RETRIEVAL_QUERY mode)
3. FAISS nearest-neighbor search → returns Top-3 semantically matching policies
4. CodeBERT runs only on these 3 policies (not all 12) → saves 75% of inference cost

WHY THIS MATTERS

3072
dims
Gemini embedding vector size

75%
less
CodeBERT inference calls vs brute-force

Top-3
policies
Retrieved per code chunk

WHY TWO LLMS?

LLaMA 3.1 8B — Fast & cheap. Generates 2-3 sentence business summaries only.

Kimi K2
— Reasoning model. Deep compliance judgment with structured JSON output.

Separating concerns = lower cost + higher quality on the hard task.

Hybrid Decision Engine

The core innovation: CodeBERT as a fast gatekeeper before calling the expensive LLM

CodeBERT Classifier
p(violation)

> 65%	35–65%	< 35%
FAST PATH	FULL REASO	FAST PATH
HIGH CONFIDENCE VIOLATION	**AMBIGUOUS ZONE**	**HIGH CONFIDENCE COMPLIANT**
CodeBERT verdict trusted. LLM asked to not re-decide. Severity assigned.	Full Kimi K2 reasoning triggered. Classifi passed as context.	CodeBERT verdict trusted. LLM confirms and explains clean code.

Output always: compliant | violations[] | explanation | severity · Prefixed with [Hybrid/high-confidence→LLM-explain] or [Hybrid/ambiguous→full-LLM]

Fine-Tuning CodeBERT with LoRA

jogi.ipynb — Model training module

CODEBERT ARCHITECTURE

- RoBERTa-based transformer pre-trained on NL + code (Python, Java, JS…)
- Input: (code_chunk, policy_text) concatenated and tokenized (max 512 tokens)
- Fine-tuned head: classifier.dense ⟶ classifier.out_proj
- Output: 2 logits ⟶ softmax ⟶ p(COMPLIANT) and p(VIOLATION)
- Binary sequence classifier: 0 = compliant, 1 = violation

LORA HYPERPARAMETERS

lora_r	16	Binary classifier — higher wastes VRAM
lora_alpha	32	Standard 2×r for stable gradients
lora_targets	Q, K	Query + Key attention matrices
lr	2e-4	LoRA standard; lower = no overfit
epochs	5	Converges on ~1000 real samples
scheduler	cosine	Smooth decay vs linear
fp16	True	Half-precision on T4 GPU
effective batch	32	16 × grad_accum of 2

Training Data & Dataset Strategy

Real datasets only — NO synthetic data. Four public HuggingFace datasets used.

SecureCode-v2

Security

`scthornton/securecode-v2`

Python code samples labeled secure/vulnerable. Mapped to policy violations.

CodeSearchNet

General

`code_search_net`

Large-scale NL+code pairs. Functions labeled by keyword-policy mapping.

BigCode

Scale

`bigcode/the-stack`

Massive Python codebase. Chunked and policy-assigned via keyword matching.

Vuln Detection

Security

`HF vulnerability datasets`

Vulnerability-labeled code. violation=1 for security-failing functions.

Policy assignment: keyword overlap (no API calls) — payment→AuditLogger policy, email→communication policy, etc.

Audit Results — Live Run

337	24	93%	9
Code Chunks Audited	Violations Found	Compliance Rate	Python Files Scanned

File	Chunks	Violations	Severity
payment/abstract_models.py	27	5	HIGH
customer/abstract_models.py	26	6	HIGH
offer/abstract_models.py	90	5	MEDIUM
checkout/utils.py	33	3	MEDIUM
payment/utils.py	19	1	MEDIUM
customer/utils.py	8	2	LOW
checkout/session.py	21	0	CLEAN ✓
offer/utils.py	3	0	CLEAN ✓

Key Design Decisions

01 AST Chunking > Line Splitting

Python's ast module extracts semantically complete functions and classes — not arbitrary line blocks. Each chunk has one clear purpose.

02 Hybrid Routing = Cost Control

High-confidence cases (>65% or <35%) bypass deep LLM reasoning. Only ambiguous cases get full Kimi K2 treatment — saving ~60% LLM calls.

03 FAISS Before CodeBERT

Running CodeBERT on all 12 policies = 12 inference passes per chunk. FAISS narrows to top-3 first → reduces classifier calls by 75%.

04 Rate-Limit Retry Logic

call_groq_with_retry() parses the exact wait time from Groq's error message and sleeps precisely that long — up to 5 attempts. Zero wasted time.

05 Graceful Degradation

If CodeBERT fails to load, the pipeline degrades to LLM-only mode automatically — same output schema, just without classifier confidence scores.

06 LoRA for Efficiency

LoRA fine-tunes only the query+key attention matrices (not all weights). 16M parameters added — full fine-tuning would require 10× more compute.

Team Contributions

Rithvik

System Architect

Slides 2, 3, 4, 10, 11

- Designed the end-to-end architecture
- Built the hybrid decision engine (fast-path + full-LLM routing)
- Implemented FAISS pipeline + Gemini embeddings integration
- AST chunking logic and code parser
- Rate-limit retry handler (call_groq_with_retry)
- Graceful fallback / LLM-only degradation mode
- Full report generation (JSON + PDF with ReportLab)
- System integration and overall codebase

Rishil

Pipeline & Integration Engineer

Slides 5, 6, 7

- Policy framework design — 12 policies across 5 domains
- LLaMA 3.1 8B summarization prompt engineering
- Kimi K2 audit prompt design and JSON output schema
- FAISS indexing and retrieval integration
- Two-LLM architecture rationale and cost analysis
- Results analysis and violation categorization

Rohan

ML Engineer — Model Fine-Tuning

Slides 8, 9

- Fine-tuned CodeBERT (jogi.ipynb) as binary violation classifier
- LoRA configuration: r=16, alpha=32, targets=Q+K
- Policy document parser (.txt / .pdf support)
- Real dataset loading pipeline (4 HuggingFace datasets)
- Keyword-based policy assignment for training pairs
- Hyperparameter tuning (cosine scheduler, fp16, grad accum)

Thank You

Policy-Aware Code Compliance Auditor

CodeBERT + LoRA | LLaMA 3.1 8B | Kimi K2 | Gemini Embed | FAISS | 93% Compliance

A hybrid AI auditing pipeline combining fine-tuned transformers, RAG retrieval, and multi-LLM reasoning to enforce business policies at scale.

Rithvik
System Architect

Rishil
Pipeline & Integration

Rohan
Model Fine-Tuning